



中国科学技术大学
University of Science and Technology of China

Introduction to RL

Huiyang Peng

School of management, USTC

2025 年 5 月 25 日



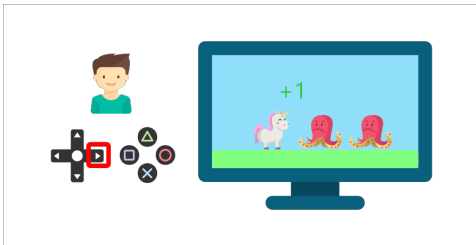
- 1 Introduction
- 2 Q-learning
- 3 Deep Q-learning Network(DQN)
- 4 References



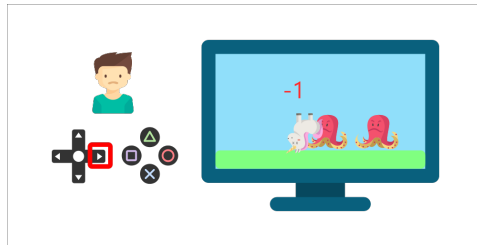
- 1 Introduction
- 2 Q-learning
- 3 Deep Q-learning Network(DQN)
- 4 References



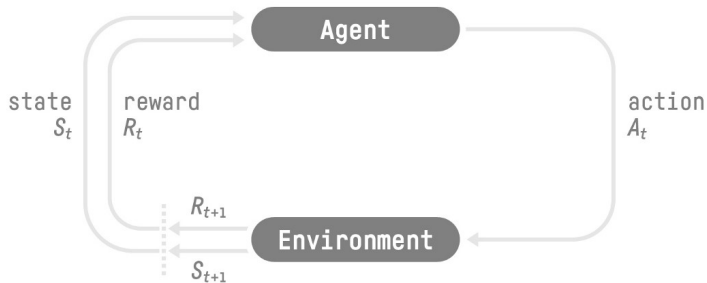
- ▶ The idea behind Reinforcement Learning is that an agent (an AI) will learn from the environment by interacting with it (through trial and error) and receiving rewards (negative or positive) as feedback for performing actions.
- ▶ Learning from interactions with the environment comes from our natural experiences.
- ▶ For instance, imagine putting your little brother in front of a video game he never played, giving him a controller, and leaving him alone.



(a) Pick a coin



(b) Touch an enemy



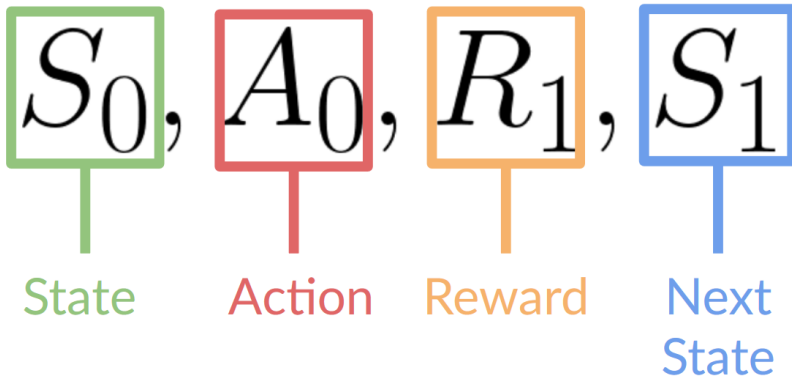


To understand the RL process, let's imagine an agent learning to play a platform game:

- ▶ Our Agent receives state S_0 from the Environment —we receive the first frame of our game (Environment).
- ▶ Based on that state S_0 , the Agent takes action A_0 —our Agent will move to the right.
- ▶ The environment goes to a new state S_1 —new frame.
- ▶ The environment gives some reward R_1 to the Agent —we're not dead (Positive Reward +1).



This RL loop outputs a sequence of state, action, reward and next state.



Why is the goal of the agent to maximize the expected return?

- ▶ Because RL is based on the reward hypothesis, which is that all goals can be described as the maximization of the expected return (expected cumulative reward).
- ▶ That's why in Reinforcement Learning, to have the best behavior, we aim to learn to take actions that maximize the expected cumulative reward.

The Reward Hypothesis

The mind's goal is to maximize the cumulative sum of a received scalar signal (reward)

Must come from outside,
not under the mind's direct control



Markov Decision Process(MDP): the agent needs only the current state to decide what action to take and not the history of all the states and actions they took before. Observations/States are the information our agent gets from the environment. It can be

- ▶ video game: a frame(a screenshot)
- ▶ trading agent: value of a certain stock

There is a *differentiation* to make between observation and state:

- ▶ State s : a complete description of the state of the world (there is no hidden information).
- ▶ Observation o : a partial description of the state.

Observation Space

State: complete description of the state of the world (no hidden information).



Observation: partial description of the state of the world.



- ▶ The reward is fundamental in RL because it's **the only feedback** for the agent.
- ▶ Thanks to it, our agent knows if the action taken was good or not.
- ▶ The **cumulative reward** at each time step t can be written as:

$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$



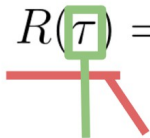
Return: cumulative reward

Trajectory (read Tau)

Sequence of states and actions

However, in reality, we can't just add them like that. The rewards that come sooner (at the beginning of the game) are more likely to happen since they are more predictable than the long-term future reward.

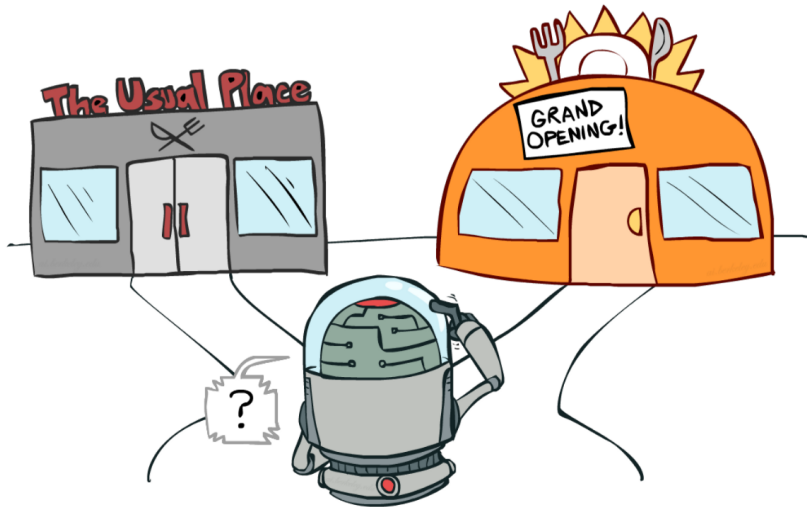
$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

 Return: cumulative reward

Gamma: discount rate

Trajectory (read Tau)

Sequence of states and actions

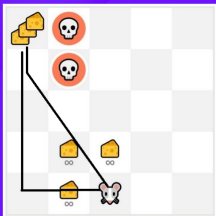


- ▶ If we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only **exploit the nearest source of rewards**, even if this source is small (exploitation).
- ▶ But if our agent does a little bit of exploration, it can **discover the big reward** (the pile of big cheese).

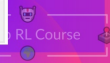
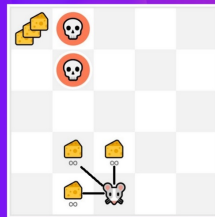


Exploration/ Exploitation tradeoff

Exploration: trying random actions in order to find more information about the environment.



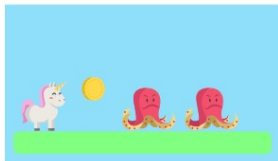
Exploitation: using known information to maximize the reward.



The Policy π : the agent's brain

There are two approaches to train our agent to find this optimal policy π^* :

- ▶ Teach the agent to learn which action to take, given the current state: Policy-Based Methods.
- ▶ Teach the agent to learn which state is more valuable: Value-Based Methods.



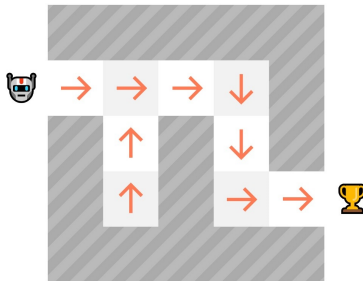
State

$\rightarrow \pi(\text{State}) \rightarrow \text{Action}$

In Policy-Based methods, **we learn a policy function directly.**

This function will define a mapping from each state to the best corresponding action.

Alternatively, it could define a **probability distribution** over the set of possible actions at that state.





We have two types of policies: **deterministic or stochastic**. The policy function can whether define a mapping from state to action or a probability distribution over the possible actions.

- ▶ Deterministic: a policy at a given state will always return the same action.

$$a = \pi(s).$$

- ▶ Stochastic: outputs a probability distribution over actions.

$$\pi(a|s) = P[A|s]$$



- ▶ In value-based methods, instead of learning a policy function, we learn a value function that maps a state to the expected value of being at that state.
- ▶ The value of a state is the expected discounted return the agent can get if it starts in that state, and then **acts according to our policy**(going to the state with the highest value).

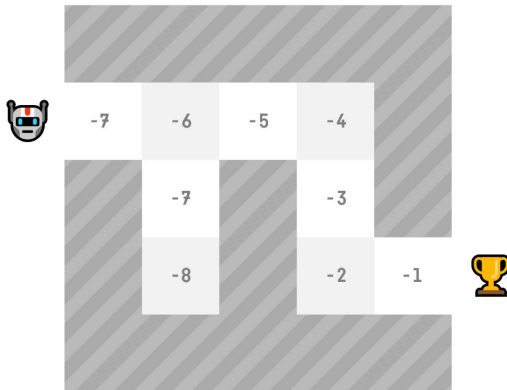
$$\underline{v_{\pi}(s)} = \underline{\mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots]} \mid \underline{S_t = s}$$

Value
function

Expected discounted return

Starting
at state s

Here we see that our value function defined values for each possible state.





- 1 Introduction
- 2 Q-learning
- 3 Deep Q-learning Network(DQN)
- 4 References

Two types of Value-Based Methods

State Value Function:

$$V_{\pi}(s) = \mathbf{E}_{\pi}[G_t | S_t = s]$$

Value of state s

Expected return

If the agent starts
at state s

And uses the policy to
choose its actions for
all time steps

For each state,
the state-value function outputs
the expected return
if the agent starts in that state
and then follows the policy forever after.



Deep RL Course

Two types of Value-Based Methods

Action Value Function:

$$Q_{\pi}(s, a) = \mathbf{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Value of state-action
pair s, a

Expected return

If the agent starts
at state s

and chooses action
 a

And then uses the
policy to choose its
actions for all time
steps

For each state and action,
the action-value function outputs
the expected return
if the agent starts in that state
and takes the action
and then follows the
policy forever after.



Deep RL Course





The Bellman equation **simplifies our state value or state-action value calculation.**

- ▶ To calculate the $V(S_t)$, we need to calculate the sum of the expected rewards.
- ▶ To calculate the $V(S_{t+1})$, we need to calculate the return starting at that state S_{t+1} .

We' re repeating the computation of the value of different states, which can be tedious if you need to do it for each state value or state-action value!

Instead of calculating the expected return for each state or each state-action pair, we can use **the Bellman equation.**

The Bellman Equation

$$V_{\pi}(s) = \mathbf{E}_{\pi} [R_{t+1} + \gamma * V_{\pi}(S_{t+1}) | S_t = s]$$

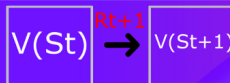
Value of
state s

Expected value of
immediate reward

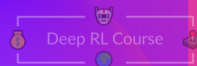
+ the discounted value of
next_state

If the agent
starts at state s

And uses the policy to
choose its actions for
all time steps



$$V(S_t) = R_{t+1} + \gamma * V(S_{t+1})$$



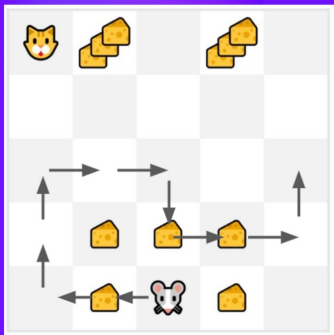


An RL agent learns by interacting with its environment. The idea is that given the experience and the received reward, the agent will update its value function or policy.

Monte Carlo and **Temporal Difference** Learning are two different strategies on how to train our value function or our policy function. Both of them use experience to solve the RL problem.

- ▶ Monte Carlo uses an entire episode of experience before learning.
- ▶ Temporal Difference uses only a step $(S_t, A_t, R_{t+1}, S_{t+1})$ to learn.

Monte Carlo Approach:



- Calculate the return G_t .

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} \dots$$

$$G_t = 1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0$$

$$G_t = 3$$

- We can now update $V(S_0)$.

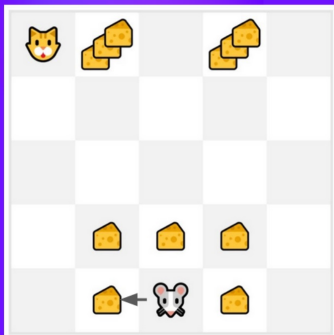
$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

$$\text{New } V(S_0) = V(S_0) + \text{lr} * [G_t - V(S_0)]$$

$$\text{New } V(S_0) = 0 + 0.1 * [3 - 0]$$

$$\text{New } V(S_0) = 0.3$$

TD Approach:



- We can now update $V(S_0)$:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New $V(S_0) = 0 + 0.1 * [1 + 1 * 0 - 0]$

The new $V(S_0) = 0.1$

So we just updated our value function for State 0.

Now we continue to interact with this environment with our updated value function.



What is Q-Learning?

Q-Learning is an

- ▶ Off-policy: using a different policy for acting (inference) and updating (training).
- ▶ Value-based method: finds the optimal policy indirectly by training a value or action-value function that will tell us **the value of each state or each state-action pair**.

that uses a

- ▶ TD approach: **updates its action-value function at each step instead of at the end of the episode**.

to train its action-value function.

We will begin with an example.



Q-Learning, Step 1

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

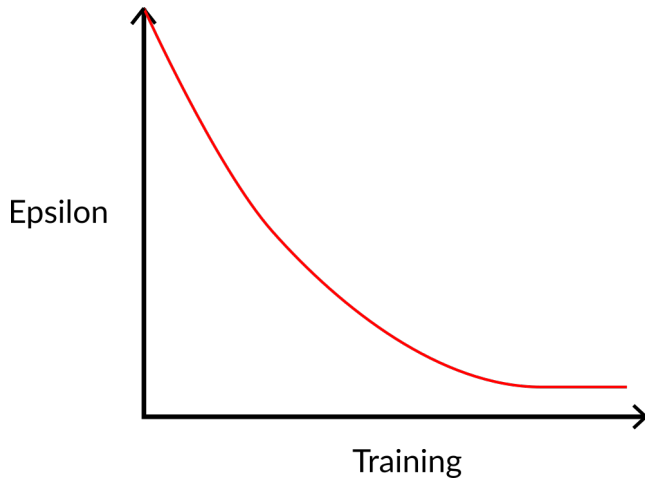
We initialize the Q-Table

Q-Learning, Step 2

Choose action A_t using policy derived from Q (e.g., ϵ -greedy)



Choose the action using ϵ -greedy policy



Q-Learning, Step 3

Take action A_t and observe R_{t+1}, S_{t+1}

Step4: Update $V(S_t)$ and $Q(S_t, A_t)$

To produce our TD target, we used the immediate reward R_{t+1} plus the discounted value of the next state, computed by finding the action that maximizes the current Q-function at the next state. (We call that bootstrap).

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New value
of state t

Former
estimation of
value of state
t

Learning Rate
Reward

Discounted value of next
state

TD Target

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New
Q-value
estimation

Former
Q-value
estimation

Learning Rate
Immediate Reward

Discounted Estimate
optimal Q-value
of next state

Former
Q-value
estimation

TD Target

TD Error

- ▶ Off-policy: using a different policy for acting (inference) and updating (training).
- ▶ On-policy: using the same policy for acting and updating.

For example, in **Q-learning**, the updating policy:

$$\gamma \max_a Q(S_{t+1}, a),$$

the acting policy:

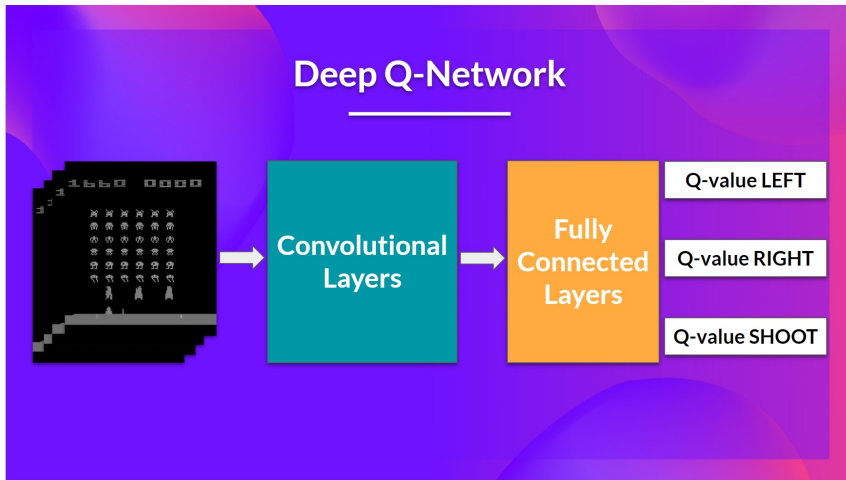
Choose action A_t using policy derived from Q (e.g., ϵ -greedy)



- 1 Introduction
- 2 Q-learning
- 3 Deep Q-learning Network(DQN)
- 4 References

We got excellent results with Q – *learning* algorithm, but these environments were relatively simple because the state space was discrete and small. For comparison, the state space in Atari games can contain 10^9 to 10^{11} states.





Algorithm 1 Deep Q-learning with Experience Replay

 Initialize replay memory $\mathcal{D}$ to capacity N

 Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
for $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for
end for

Q-Target

$$y_j = r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-)$$

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

Immediate
Reward

Discounted Estimate
optimal Q-value
of next state

TD Target

Q-Loss

$$y_j - Q(\phi_j, a_j; \theta)$$

$$[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Immediate
Reward

Discounted Estimate
optimal Q-value
of next state

Former
Q-value
estimation

TD Target

TD Error



After DQN was introduced in Nature, many people must have been thinking about how to improve it. So, what are the problems with DQN?

- ▶ Is the calculation of the target Q-value accurate? Is it problematic to calculate everything using the max Q value?
- ▶ Is the random sampling method effective? Ideally, the importance of different samples is not the same.
- ▶ Q-values represent the value of a state-action pair, so would evaluating the action value independently be more accurate?



- ▶ Double DQN: Remove upward bias caused by $\max_a Q(s, a; \theta)$
 - ▶ Current Q-network θ is used to select actions
 - ▶ Older Q-network θ^- is used to evaluate actions

$$l = (r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) - Q(s, a; \theta))^2$$

- ▶ Prioritised replay: Weight experience according to surprise
 - ▶ Store experience in priority queue according to DQN error

$$|r + \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)|$$

- ▶ Duelling network: Split Q-network into two channels
 - ▶ Action-independent value function $V(s; v)$
 - ▶ Action-dependent advantage function $A(s, a; \theta)$

$$Q(s, a) = V(s; v) + A(s, a; \theta)$$



- 1 Introduction
- 2 Q-learning
- 3 Deep Q-learning Network(DQN)
- 4 References



1. Huggingface. Deep RL Course. <https://huggingface.co/learn/deep-rl-course/>
2. UC Berkeley CS188 Intro to AI – Course Materials.
3. Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." arXiv preprint arXiv:1312.5602 (2013).
4. Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).
<https://doi.org/10.1038/nature14236>
5. Tutorial: Deep Reinforcement Learning. ICML 2016, David Silver, Google DeepMind



Thanks!